W3D1

Created time	October 21, 2026 5:54 PM

Q1

```
import pandas as pd

source = pd.DataFrame({
"product_id": [101, 102, 103, 104],
"price_gbp": ["25.50", "invalid", "12.00", "40.00"],
})

working = source.copy()

working["price_gbp"] = pd.to_numeric(
working["price_gbp"],
errors="coerce",
)

clean_products = working[
working["price_gbp"].notna()
].copy()

rejected_products = working[
working["price_gbp"].isna()
].copy()

print("Clean rows:")
print(clean_products)

print("Rejected rows:")
print(rejected_products)
```

Q2.

```
import pandas as pd

source = pd.DataFrame({
"order_id": [201, 202, 203],
"amount_gbp": ["45.50", "30.00", "bad"],
})
```

```
raw_orders = source.copy()

clean_orders = raw_orders.copy()

clean_orders["amount_gbp"] = pd.to_numeric(
clean_orders["amount_gbp"],
errors="coerce",
)

clean_orders = clean_orders[
clean_orders["amount_gbp"].notna()
].copy()

print("Raw:")
print(raw_orders)

print("Clean:")
print(clean_orders)
```

Q3.

Requirement	Choice	Reason
Remove invalid data before target	ETL	Transformation happens first
Keep raw data first	ELT	Source data is loaded before transformation
Strong target SQL capability	ELT	Transformation can happen in the target
Remove sensitive fields before loading	ETL	Sensitive values are changed before the target receives them

Q4. C. Extract → Transform → Load

Q5.

```
batches = {
"batch_1": [10, 20, 30],
"batch_2": [40, 50],
"batch_3": [60, 70, 80],
}

for batch_name, values in batches.items():
print(
```

```
batch_name,
"rows =", len(values),
"total =", sum(values),
)
```

Q6.

1. Supplier sends one file every night. — Batch
2. Payment events arrive continuously and need a result within seconds. — Streaming
3. Finance produces one monthly report. —- Batch
4. Website click events arrive throughout the day.— Streaming

Q7.

```
import pandas as pd

source = pd.DataFrame({
"order_id": [301, 302, 303],
"city": [" London ", None, "Leeds"],
"amount_gbp": [45.50, 20.00, -5.00],
})

raw_orders = source.copy()

clean_orders = source[
source["city"].notna()
& (source["amount_gbp"] > 0)
].copy()

clean_orders["city"] = (
clean_orders["city"].str.strip()
)

print("Raw rows:", len(raw_orders))
print("Clean rows:", len(clean_orders))

print(raw_orders)
print(clean_orders)
```

Q8.

Which requirement is the strongest reason to use streaming?
A. Fraud events must be checked within seconds.

Q9.

A. Rename source columns to standard names. — Staging
B. Join orders to customers. —- Transformation
C. Calculate daily revenue by city. —- Analytics
D. Convert an amount column from text to numeric. — Staging
E. Create a report-ready monthly sales table. — Analytics

Q10.

```
import pandas as pd

source = pd.DataFrame({
"OrderID": [401, 402, 403],
"CustomerID": [1, 2, 1],
"Amount": ["45.50", "30.00", "20.00"],
})

staging = source.rename(columns={
"OrderID": "order_id",
"CustomerID": "customer_id",
"Amount": "amount_gbp",
}).copy()

staging["amount_gbp"] = pd.to_numeric(
staging["amount_gbp"],
errors="raise",
)

print(staging)
```

Q11.

```
import pandas as pd

orders = pd.DataFrame({
"order_id": [501, 502, 503, 504],
"city": ["London", "Leeds", "London", "Manchester"],
"amount_gbp": [45.50, 30.00, 62.25, 50.00],
```

```
"status": ["complete", "complete", "complete", "cancelled"],
})

completed = orders[
orders["status"] == "complete"
].copy()

report = (
completed
.groupby("city", as_index=False)
.agg(
completed_revenue_gbp=(
"amount_gbp",
"sum",
)
)
)

print(report)
```

Q12. D. Rename and standardise source columns before deeper transformation.

Q13.

```
load_source
↓
clean_orders
↓
build_daily_sales
↓
publish_report
```

Q14.

Failure	Retry useful?	Reason
Temporary network timeout	Yes	The condition may clear
Python syntax error	No	Code must be corrected
Short database outage	Yes	The service may recover
Required column removed	No	Input or code must be corrected

Q15.

1. Confirm the source data exists for all three dates.
2. Check whether any partial output already exists.
3. Process 2026-08-05.
4. Validate counts and totals.
5. Process 2026-08-06.
6. Validate counts and totals.
7. Process 2026-08-07.
8. Validate counts and totals.
9. Confirm that rerunning a date will not duplicate data.

Q16. Why is a dependency needed between `clean_orders` and `build_daily_sales`?

B. To ensure the sales summary waits for clean data.

Q17.

```
import pandas as pd

source = pd.DataFrame({
"order_id": [601, 602, 603, 604],
"city": ["London", None, "Leeds", "Manchester"],
"amount_gbp": ["45.50", "30.00", "bad", "50.00"],
"status": ["complete", "complete", "complete", "cancelled"],
})

raw_orders = source.copy()

working = raw_orders.copy()

working["amount_gbp"] = pd.to_numeric(
working["amount_gbp"],
errors="coerce",
)

clean_orders = working[
working["city"].notna()
```

```
& working["amount_gbp"].notna()
].copy()

completed = clean_orders[
clean_orders["status"] == "complete"
].copy()

daily_sales = (
completed
.groupby("city", as_index=False)
.agg(
revenue_gbp=("amount_gbp", "sum")
)
)

print("Raw:")
print(raw_orders)

print("Clean:")
print(clean_orders)

print("Daily sales:")
print(daily_sales)
```

Q18.

```
load_source
↓
clean_orders
↓
build_daily_sales
↓
publish_report
```

Q19.

Using append is risky because rerunning 2026-08-05 may duplicate rows, causing incorrect totals and reports.

Before rerunning, the team should check:

- What was already written for that date.

- Whether duplicates exist in the target table.
- The write strategy — delete and replace that date's data, or use an idempotent upsert.
- Record counts and totals after rerunning to verify correctness.

Q20.

Layer	Input	Main work	Output	Validation
Raw	Daily order file	Keep received data	Raw orders	Input row count
Staging	Raw orders	Standardise columns and types	Staged orders	Required columns
Transformation	Staged orders	Clean, join, apply business rules	Trusted orders	Valid/rejected counts
Analytics	Trusted orders	Aggregate for reporting	Daily sales	Revenue reconciliation